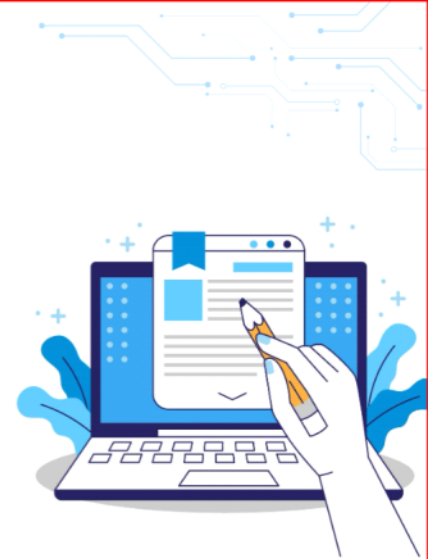


Working with LLMs: APIs, SDKs, Parameters and Open-Source Models

Topics

- e! LLM Landscape: OpenAI, Anthropic, Gemini, Mistral API, LLaMA
- e! Core Capabilities: Summarization, Q&A, Translation, Code Generation
- e! Key Configuration Parameters: Temperature, Top_P, Max_Tokens, Stop Sequences
- e! Inference Techniques: Sampling, Beam Search, Greedy Decoding
- e! Efficient Use of Tokens and Context Window
- e! Calling Tools
- e! Functions With LLMs
- e! Deployment Considerations for Open-Source LLMs (Local, Cloud, Fine-Tuning)
- e! Rate Limits, Retries, Logging
- e! Understanding Cost, Latency, and Performance and Calculating via Code



Learning Objectives

By the end of this module, you will be able to:

- e! Identify major LLM providers and their core offerings
- e! Describe key capabilities supported by modern LLMs
- e! Explain how core configuration parameters affect model output
- e! Compare different inference techniques used in LLMs
- e! Apply strategies to optimize token usage and manage context limits
- e! Implement function calling with structured outputs in LLMs
- e! Evaluate deployment strategies for open-source LLMs
- e! Analyze cost and latency trade-offs using code-based measurements

LLM Landscape: OpenAI, Anthropic, Gemini, Mistral API, LLaMA

Introduction to LLM Landscape

LLM Landscape refers to the competitive ecosystem of Large Language Model providers offering commercial APIs and open-source models with varying AI capabilities, pricing, and deployment options.

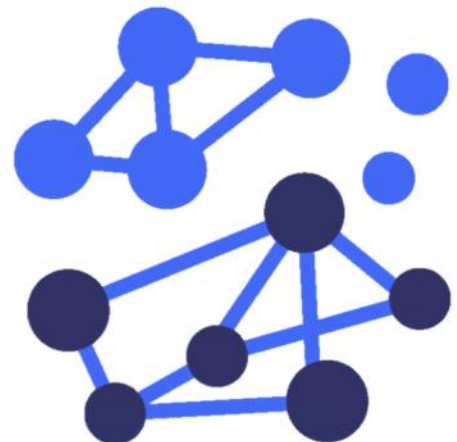
- 1 Diverse Provider Ecosystem:** OpenAI, Anthropic, Google, Mistral AI, and Meta.
- 2 Cost Optimization Strategies:** Different pricing models, per-token, subscription, and infrastructure-based.
- 3 Customizable Behavior:** Different APIs, context limits, multimodal support, and hosting options.
- 4 Compliance:** Providers follow distinct data policies and regional regulations.



What are LLM APIs?

APIs (Application Programming Interfaces) are tools that enable developers to interact with large language models (LLMs) for tasks such as text generation, summarization, translation, and more.

- 01 Pre-trained Models:** Utilize available LLMs.
- 02 Customizability:** Fine-tune prompts to suit particular tasks.
- 03 Scalability:** Handle several requests at the same time.
- 04 Ease of Use:** REST API calls, often supported by SDKs.



Types of LLM APIs



01

General Purpose LLM APIs:

Ensure availability of flexible LLMs for various tasks.

02

Specialized LLM APIs:

Concentrate on particular tasks or areas such as code generation or medical applications.

03

Fine-Tunable LLM APIs:

Allow customization of the base models using user-specific data for niche applications.

04

Multi-Modal APIs:

Expand LLM abilities to manage various data types such as text, images, or audio.

Overview of OpenAI



A leading cloud-based interface offering access to OpenAI's flagship multimodal LLMs (GPT-3.5, GPT-4, GPT-4o), ideal for text, code, image, and audio generation and understanding.

01

Model Variety: GPT-3.5 Turbo, GPT-4 with 8 K/32 K/128 K context windows, and GPT-4o for multimodal interaction.

02

Multimodality: Accepts and outputs text, images, audio, and video—great for rich interactions.

03

Developer Ecosystem: Well-documented REST API, multiple SDKs, supports function calling, streaming, and plug-ins.

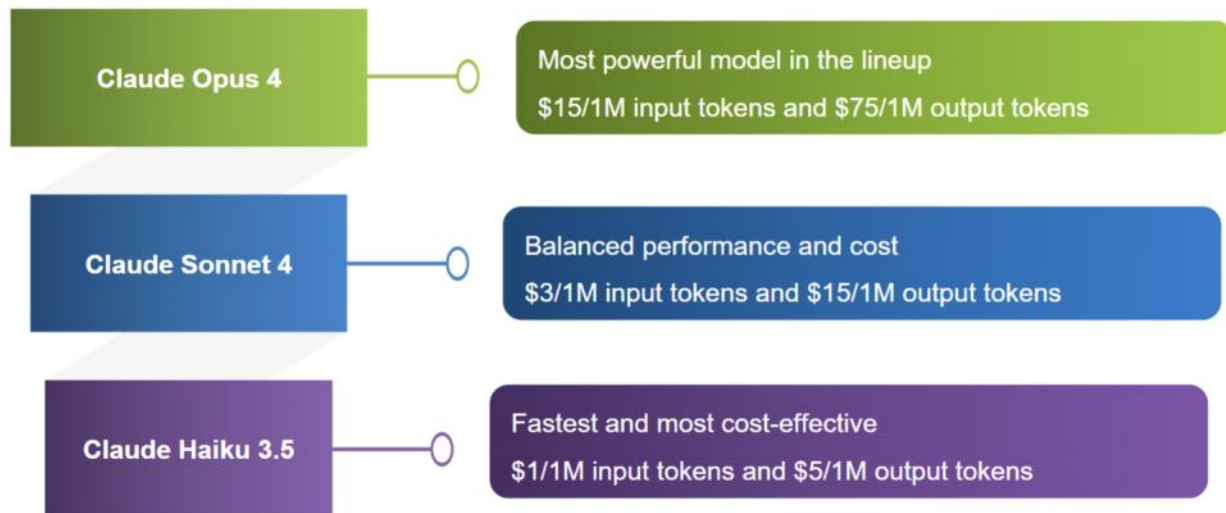
04

Scalability: From lightweight GPT-3.5 Turbo to powerful GPT-4/GPT-4o for heavy-duty workloads.



ChatGPT

Anthropic Claude - Safety-First AI



Anthropic Capabilities



Introduction to Google Gemini

Google DeepMind's multimodal LLM accessible via Vertex AI/ AI Studio, capable of reasoning across text, images, audio, video, and code.

01

Models: Nano, Pro, Ultra; Gemini 2.0 Pro supports 2M tokens.

02

Multimodal: Handles image, audio, and video inputs with advanced outputs.

03

Developer Tools: Supports Python, JS, Go SDKs, and Vertex AI.

04

Evolution: Gemini 2.5 Pro and Live (voice/video) announced at Google I/O 2025.

The Gemini logo, featuring the word "Gemini" in a blue and purple gradient font, with a small four-pointed star above the 'i'.

Key Features of Gemini's Architecture

Cross-modal understanding: Handles various data types within context.

Unified transformer framework: Facilitates multimodal learning.

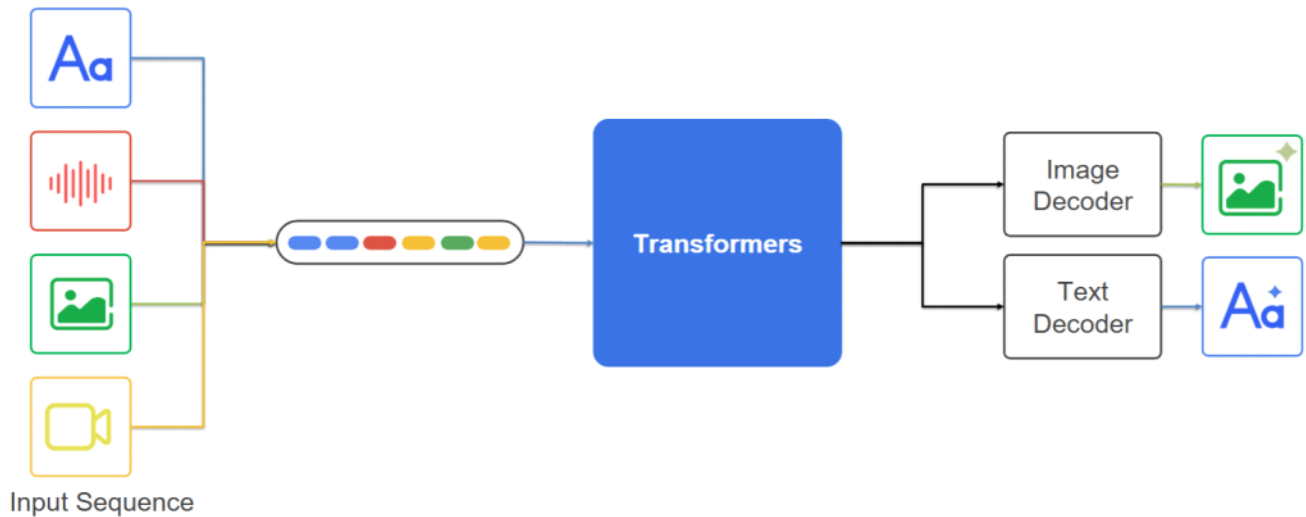
Scalability for complex tasks: Manages high-dimensional data effectively.

Enhanced reasoning: Utilizes contextual connections across modalities.



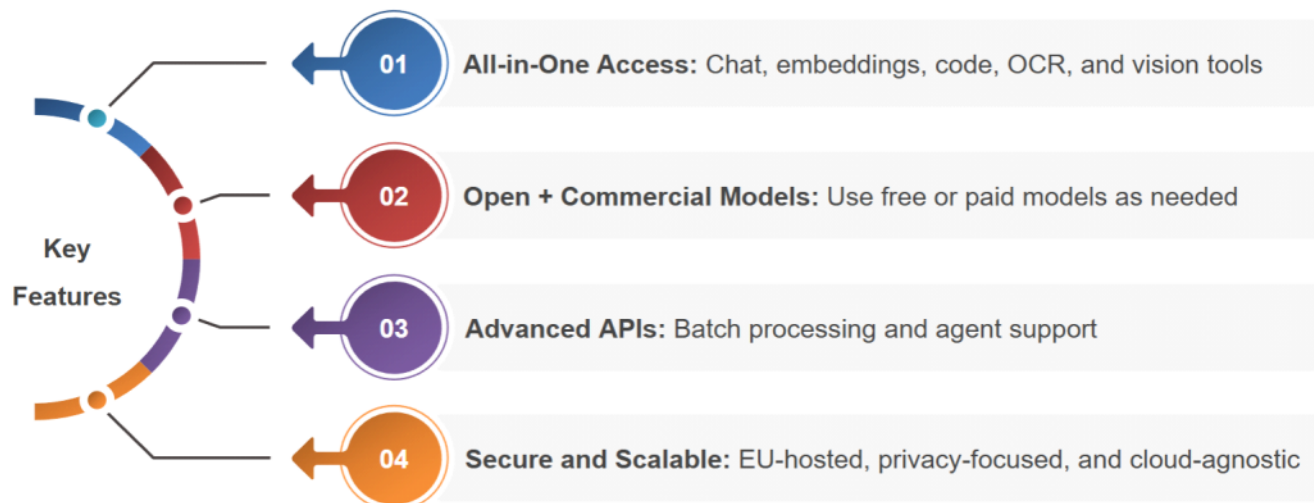
Gemini's Multimodal Architecture

Gemini processes text, audio, image, and video inputs through a unified Transformer model to generate coherent text or image outputs.



What is Mistral API?

Mistral API offers access to powerful open-source and commercial LLMs for chat, vision, OCR, code, and more.

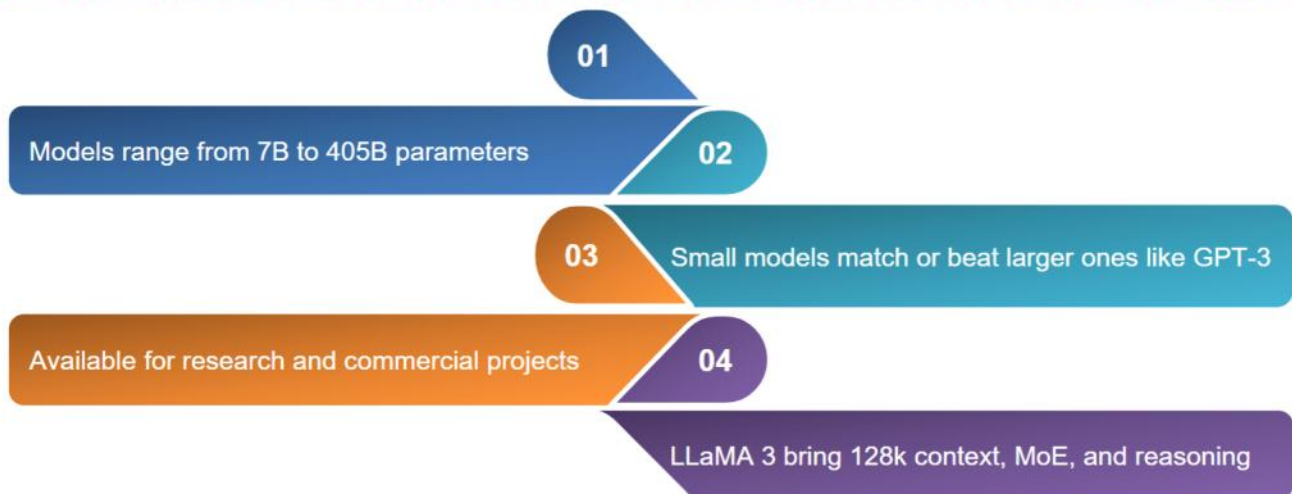


Types of Mistral Model Suite



Introduction to Llama (Meta AI)

LLaMA (Large Language Model Meta AI) is Meta's family of open-source LLMs designed for both research and commercial use.



Model Sizes of Llama

01

LLaMA 1: 7B, 13B, 32.5B, 65B.

02

LLaMA 2: 7B, 13B, 70B, Finetuned, Quantized.

03

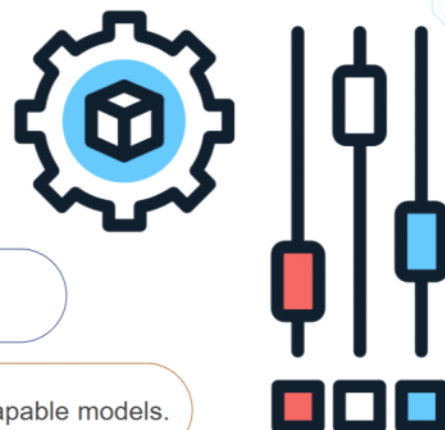
LLaMA 3 (2024): 8B, 70B, 405B—with 128K token context.

04

LLaMA 3.2 & 3.3: Include smaller (1B, 3B, 11B, 90B) and vision-capable models.

05

LLaMA 4 (Apr 5, 2025): Mixture-of-Experts - Scout & Maverick, plus upcoming Behemoth.



Model Architecture of Llama



Decoder-only transformer architecture.



Expanded tokenizer vocabulary (128K tokens).



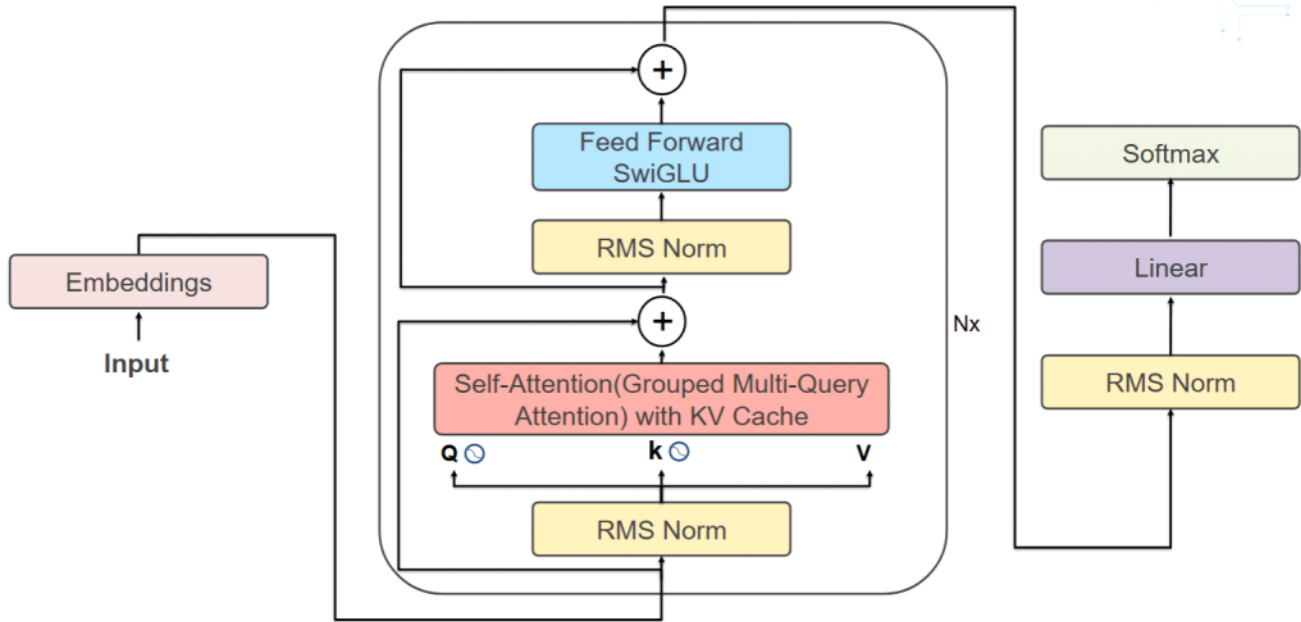
Grouped Query Attention (GQA) for efficiency.



Trained on sequences up to 8,192 tokens.



Model Architecture of Llama (Contd.)



Demo 1: API Calls with Gemini

Core Capabilities: Summarization,
Q&A, Translation, Code Generation

Introduction to Core LLM Capabilities

LLM Capabilities refer to the fundamental natural language processing tasks that Large Language Models excel at, including text summarization, question answering, language translation, and code generation.

Key Benefits



Universal Language Understanding



Task Versatility



Scalable Performance



Domain-Driven Utility

Text Summarization with LLMs

LLM Summarization is the ability to condense long-form text into shorter, coherent summaries while preserving key information and main ideas, supporting both extractive and abstractive summarization approaches.

01

Helps in quickly understanding lengthy documents or articles.

Extractive keeps the original words; Abstractive rewrites in words.

02

03

Useful in legal, medical, research, and news domains.

Prompts can be in specify length, style, or purpose.

04



Popular Tools and APIs for Summarization



Intelligent Question Answering Systems

Q&A is the ability of LLMs to answer user questions based on general knowledge or specific documents provided (contextual Q&A).

Q&A System Types

Closed-domain
Answering questions within specific knowledge areas.

Conversational
Multi-turn dialogue with context retention.



Open-domain
General knowledge questions across multiple domains.

Document-based
Processing complex queries across various domains.

Tools and APIs for Q&A Systems

OpenAI GPT-4

Comprehensive factual and reasoning-based Q&A.

Claude 4

Constitutional AI ensures safer, more reliable answers.

Gemini Pro

Powerful AI integration with a focus on search and analysis.

Mistral Instruct

Helps in fine-tuned for instruction-following Q&A.



Multilingual Translation with LLMs

LLM Translation is the ability to convert text from one language to another while preserving meaning, context, and cultural nuances, supporting both direct translation and cross-lingual understanding tasks.

Key Points:

LLMs offer multilingual support out-of-the-box (100+ languages).

Useful for global applications: chatbots, support, and localization.

Can handle low-resource languages with newer models.

Custom prompts can ensure domain-specific tone and context.

Tools and APIs for Multilingual Translation

OpenAI GPT-4

High-accuracy translations across major languages.

Claude 3 Opus

Supports translation with cultural context preservation.

Gemini 2.5 Pro

Strong multilingual understanding.

Mistral

Lightweight translations for real-time applications.



AI-Powered Code Generation

LLM Code Generation is the ability to automatically create, complete, and debug code across multiple programming languages based on natural language descriptions, comments, or partial code snippets.

01

Supports code in multiple languages: Python, JS, Java etc.

02

Enhances productivity in IDEs via extensions.

03

Can refactor, comment, or explain existing code.

04

Can be fine-tuned to generate unit tests, APIs, or SQL queries.



Code Generation Tools

Can you list tools
used for
generating code?



OpenAI Codex - Best for general-purpose code generation.

Gemini Pro - Optimized for software development tasks.

Claude 3 Sonnet/Opus - Handles longer code contexts.

Tabnine / CodeWhisperer - Dedicated code assistants.

Specialized Tools - GitHub Copilot, Replit, and Cursor.

Key Configuration Parameters: Temperature, Top_P, Max_Tokens, Stop Sequences

What are Temperature Parameter in LLMs?

Temperature is a parameter that controls the randomness and creativity of LLM outputs by adjusting the probability distribution of token selection.



Temperature < 1

More focused and
predictable responses
great for accuracy.

01



Temperature = 1

Normal behavior, no
adjustments to
creativity or precision.

02



Temperature > 1

More creative and
varied outputs - ideal
for brainstorming.

03

Use Cases for LLM Temperature Modeling

Customer Interaction

Customize tone to match brand personality and user expectations.

Temperature Range: 0.4–0.7

Creative Writing

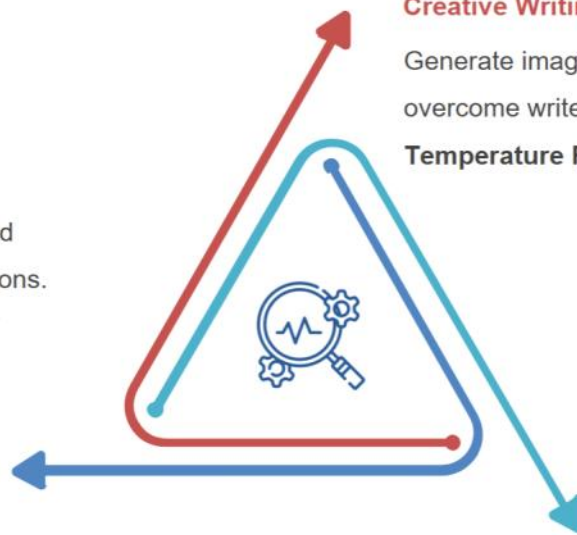
Generate imaginative, fresh content to overcome writer's block or spark ideas.

Temperature Range: 0.8–1.0+

Technical Documentation

Ensure precision, consistency, and factual accuracy.

Temperature Range: 0.2–0.4



Top_P - Precision Token Selection

Top_P is a parameter that controls token selection by considering only the smallest set of tokens whose cumulative probability exceeds the Top_P threshold, providing more nuanced control over output diversity.

Effect On Output:

Low P
(e.g., 0.1)

Considers only the most likely tokens, resulting in more focused output.

High P
(e.g., 0.9)

Considers more tokens, resulting in richer context understanding and more diverse output.



Ideal Scenarios for Top P

01

Structured and Factual Tasks

Use low P (0.8–0.9) for focused, reliable outputs (e.g., technical responses, documentation).

02

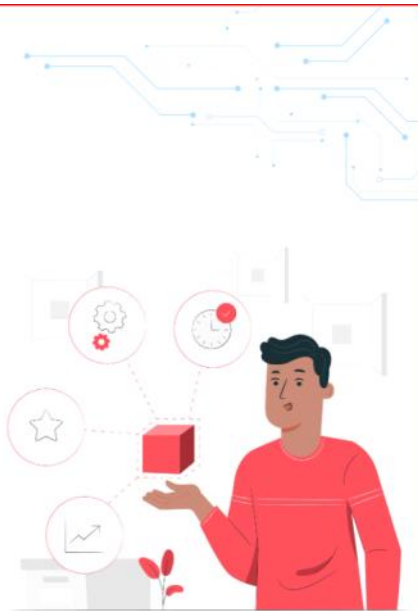
Everyday Conversational AI

Use mid-range P (0.9–0.95) for natural, varied dialogue without straying off-topic.

03

Creative Content Generation

Use high P (0.95–0.99) to encourage creative language and new ideas while staying coherent.



Overview of Max_Tokens Parameter

It is a parameter that sets the maximum number of tokens the model can generate in a single response, serving as both a cost control mechanism and a way to ensure outputs fit within desired length constraints.

Key Points:



Output tokens typically cost 3-5x more than input tokens.

Set conservative Max_Tokens for cost-sensitive applications.

Implement dynamic Max_Tokens based on query complexity.

01

02

03

Strategic Length Planning of Tokens

01

Short Responses

50-150 tokens for summaries, quick answers, status updates.

02

Medium Responses

200-500 tokens for explanations, analysis, technical guidance.

03

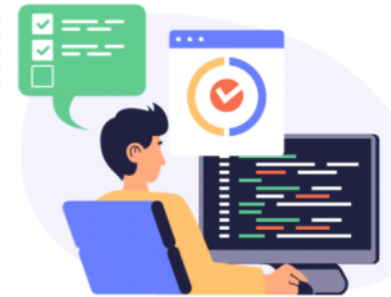
Long Responses

500-2000 tokens for detailed content, documentation, comprehensive reports.

04

Maximum Utilization

Use remaining context window for complex tasks, research.



Introduction to Stop Sequences in LLMs

Stop Sequences are predefined strings or tokens that instruct the LLM to immediately cease generation when encountered, providing precise control over output format, structure, and natural termination points.

Format Enforcement: Ensure consistent output structure.

Cost Control: Prevent unnecessary token generation.

Multi-turn Conversations: Define clear speaker boundaries.

Template Filling: Stop at specific markers in forms or documents.



Why Stop Sequences Matter?



Why Are Stop Sequences Important?

Prevent Run-Ons: Ensures output stops at the intended point (e.g., end of a sentence or list item).

Maintain Structure: Helps enforce formatted responses for easy integration.

Enhance Safety: Blocks generation of harmful or irrelevant content before it begins.



Demo 2: Tuning Parameters for Text Generation

Inference Techniques: Sampling, Beam Search, Greedy Decoding

Introduction to Inference Techniques

They are methods used by LLMs to generate text by selecting tokens at each step from probability distributions, which are critical for controlling output quality, diversity, coherence, and computational efficiency.

Different techniques produce varying levels of output quality.

Quality Control

Diversity vs Consistency

Balance between creative and predictable outputs.

Balance between inference speed, cost and output quality.

Performance Trade-offs

Use Case Optimization

Different techniques suit different applications.

What are Sampling Techniques?

Sampling methods introduce controlled randomness in token selection by choosing from probability distributions rather than always selecting the most probable token, enabling diverse and creative text generation.

Key Benefits of Sampling

1

Creativity

Introduces controlled randomness for diverse outputs.

2

Exploration

Discovers unexpected but valid continuations.

3

Human-like

More natural, less predictable text generation.



Types of Sampling Techniques

Pure Random Sampling

Sample from full vocabulary.

01

Temperature Sampling

Control randomness with temperature parameter.

02

Top-k Sampling

Sample from k most likely tokens.

03

Top-p Sampling

Sample from cumulative probability threshold.

04

Greedy Search Overview

Greedy Search is the simplest decoding method for language models. At each time step, it selects the single most probable token given the current context.

Key Points

01

Deterministic: Always picks the highest probability token, giving the same result each time.

02

No Lookahead: Chooses the best token step-by-step without planning ahead.

03

Not Always Best Overall: May miss better full sequences by only following one path.



Beam Search: Using Conditional Probability

Beam Search is a generalization of greedy decoding that explores multiple likely sequences by maintaining a fixed number (beam width K) of candidate sequences at each step.

Key Aspects:

Maintains K most probable sequences instead of just one.

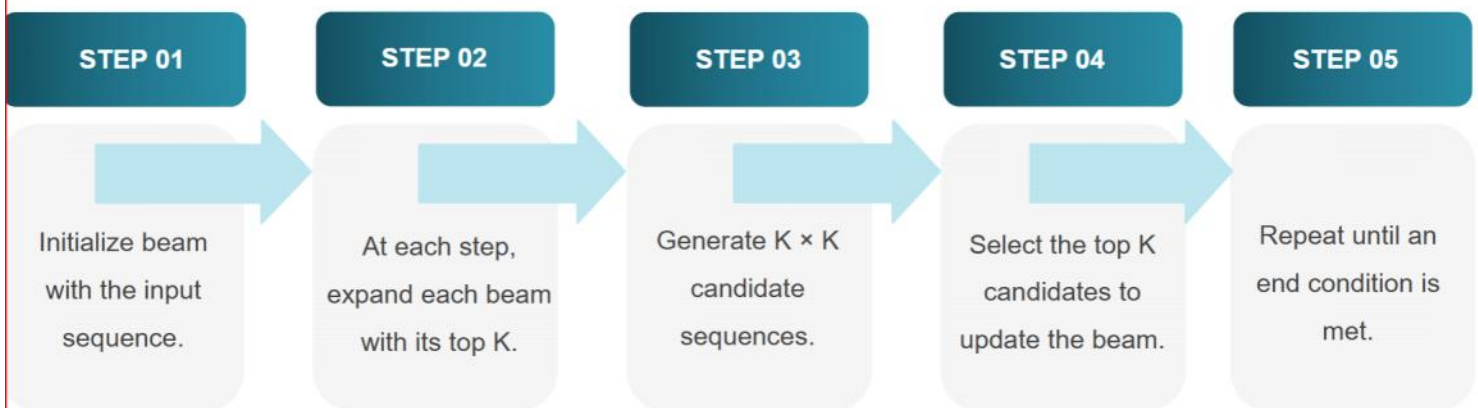
Enables the model to “look ahead” and avoid locally optimal choices.

Produces higher-quality text than greedy search due to broader exploration.

Trade-off: Increased accuracy vs. higher computational cost as K increases.



How Beam Search Works?



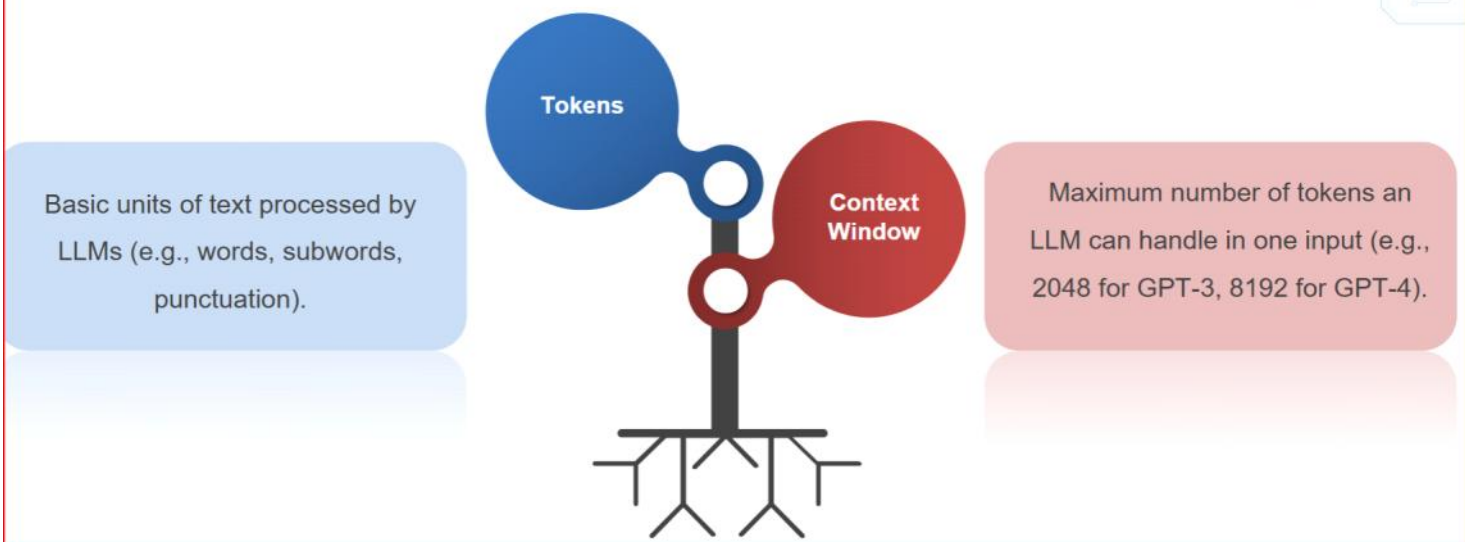
Comparison of Techniques



	Technique	Speed	Output Quality	Best For
01	Greedy Decoding	Fast	Lower	Quick drafts, simple tasks
02	Sampling	Moderate	Variable	Creative writing, ideation
03	Beam Search	Slower	Higher	Coherent text, summaries

Efficient Use of Tokens and Context Window

Introduction to Tokens and Context Window



Techniques to Manage the Context Window

Sliding Window

Process overlapping segments
Maintains context between chunks.

01

Prompt Engineering

Include only essential info in prompts
Break complex tasks into smaller steps.

02

Chunking & Summarizing

Split large text into smaller chunks
Summarize chunks to retain key ideas.

03

Dynamic Prompting

Update prompts based on model responses
Remove outdated or less relevant info.

04



Efficient Token Usage Techniques

Use Sliding Window

Break long text into smaller
overlapping chunks.

03

Summarize Past Context

Replace long histories with brief
summaries.

04

Limit Output Length

Set max_tokens to avoid
long responses.

02

Select Relevant Context Only

Include only what the model
needs to know.

05

Keep Prompts Concise

Remove unnecessary words
and be direct.

01

Use Compact Tokenization

Leverage subword or
compressed formats.

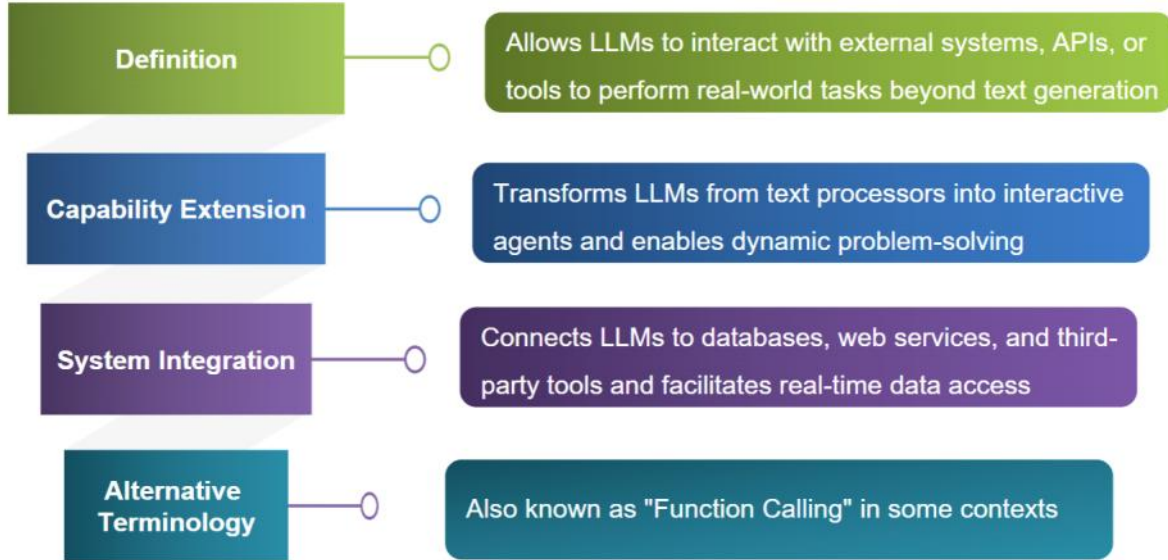
06



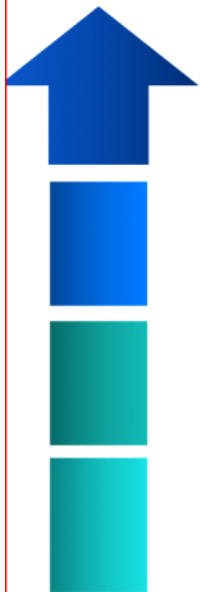
Demo 3: Token Usage Optimization

Calling Tools

Introduction to Tool Calling



Why Tool Calling Matters?



Agent Capabilities: Autonomously selecting and using functions or services to handle tasks on your behalf.

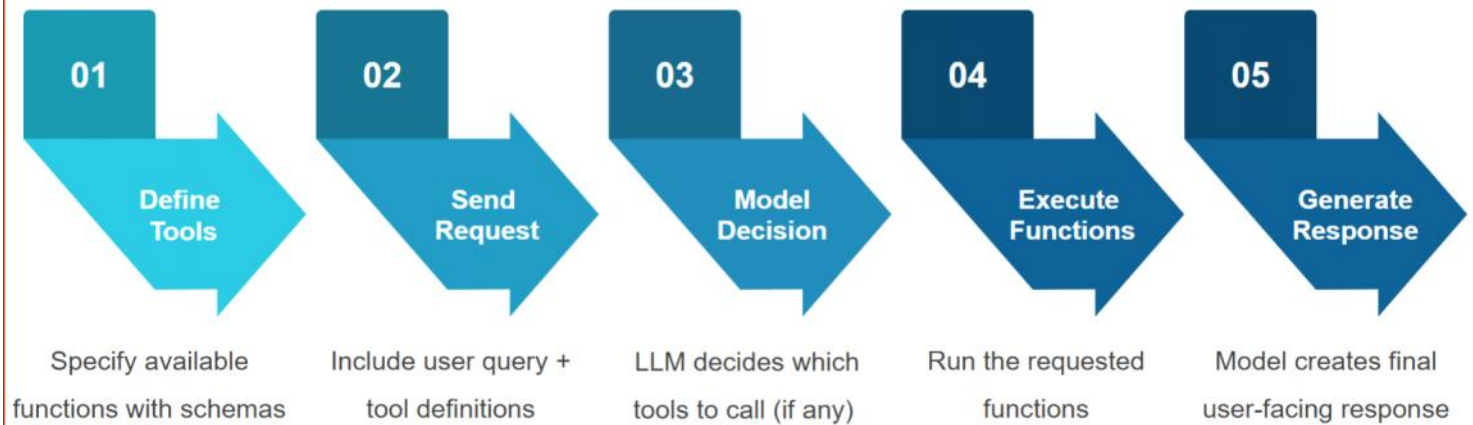
Language-to-Action Bridge: Bridges the gap between language understanding and action.

Intelligent System Development: Enables building intelligent agents and assistants.

Real-World Application Power: Powers real-world applications like chatbots with external capabilities.



How Tool Calling Works?



Functions With LLMs

Introduction to Functions in LLMs

The ability to reliably connect LLMs to external tools to enable effective tool usage and interaction with external APIs.

Why to Use Functions?

- 1 Enable dynamic interactions with external systems (e.g., APIs, databases).
- 2 Enhance accuracy with real-time information access and analysis.
- 3 Support complex workflows by chaining multiple functions in sequence.



Use Cases of Functions in LLMs

01. Conversational Agents: Enhance chatbots by calling APIs or databases for accurate, dynamic responses.

02. Natural Language Understanding: Convert text to structured JSON, extract entities, sentiment, and keywords.

03. API Integration: Turn natural language into valid API calls to fetch data or perform actions.

04. Information Extraction: Pull specific facts, references, or news from large text inputs.



Core Workflow of Functions



name, description, parameters via JSON-schema.

Include definitions in the prompt payload.

Whether to call a function and outputs JSON with arguments.

Executes the function with generated args.

Return results to the LLM to craft the final response.

Deployment Considerations for Open-Source LLMs

Local Deployment of LLMs

Running LLMs on-premises or in private infrastructure to maximize data control, customization, and speed.

Popular Local Deployment Tools

Ollama: User-friendly CLI and Python API interface.

LM Studio: GUI-based model management.

GPT4All: Cross-platform local AI chat interface.

Jan: Open-source alternative to ChatGPT.



Local Deployment Advantages

Local deployment offers several compelling benefits that make it attractive for organizations prioritizing control, privacy, and long-term cost management.

Privacy:

Complete data control,
no external API calls

Cost-Effective:

No ongoing API fees
after initial setup

Longevity:

Models stay relevant
unlike hosted services

Offline Capability:

No internet
dependency

Cloud Deployment Strategies

Cloud Deployment in the context of LLMs refers to running language models on remote servers managed by cloud providers (e.g., Azure, Google Cloud, AWS).

Key Features:

Easy Integration: Use APIs to embed models like GPT-4 and Gemini seamlessly.

1

Scalable Infrastructure: Auto-adjusts to meet changing workload demands.

2

Latest Innovations: Always up-to-date with cutting-edge model versions.

3

No Infra Overhead: Eliminates the need for local hardware or maintenance.

4

Rate Limits, Retries, Logging

Rate Limits, Retries, and Logging Overview

Rate Limits:

Restrictions on the number of API requests allowed in a given time frame to manage server load and ensure fair usage.

Retries:

Automatically resending failed requests due to transient issues like network errors or timeouts.

Logging:

Recording events, errors, and data points for debugging, monitoring, and understanding API interactions.

Common Rate Limit Types

1. RPM (Requests/Min):

Max API calls per minute

2. TPM (Tokens/Min):

Max tokens per minute

3. RPD (Requests/Day):

Daily request limit

4. Concurrent Requests:

Max simultaneous API calls allowed

5. Context Window:

Max tokens in a single conversation

6. Model-Specific Limits:

Vary by model (GPT-4 has stricter limits than GPT-3.5)



Why Logging Matters for LLM Applications?

The structured nature of LLM interactions often benefits from specialized logging approaches beyond plain text logs.

What to Log?

1

Input Prompts: User queries and system messages

Model Responses: Generated text and metadata

2

3

API Metadata: Model version, parameters, timestamps

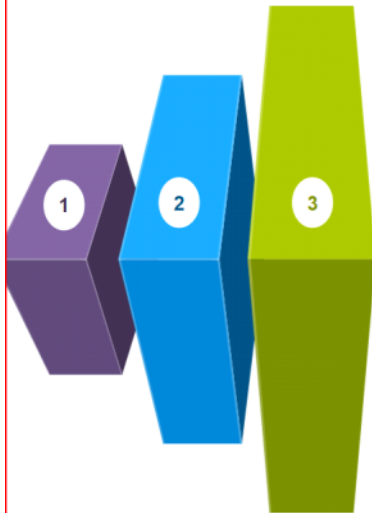
Performance Metrics: Response time, token usage, costs

4



Understanding Cost, Latency, and Performance and Calculating via Code

Overview of Cost, Latency, and Performance



Cost:

Financial expense of using LLM APIs, driven by model choice, token usage, and additional features.

Latency:

Time delay from sending a request to receiving a response, vital for real-time applications.

Performance:

Includes throughput, response quality, and resource efficiency.



Factors Affecting Cost

01

Model Selection: Advanced models are pricier than simpler ones.

02

Token Usage: Costs scale with input and output tokens processed.

03

Infrastructure: GPU or TPU usage increases cost.

04

Caching and Batching: Reuse and batching reduce token costs.



Calculating Cost via Code

Code Sample:

```
1 def calculate_cost(input_tokens, output_tokens, model="gpt-4"):
2     if model == "gpt-4":
3         input_rate = 0.03 / 1000 # $0.03 per 1k tokens
4         output_rate = 0.06 / 1000 # $0.06 per 1k tokens
5     else: # e.g., gpt-3.5-turbo
6         input_rate = 0.01 / 1000
7         output_rate = 0.02 / 1000
8     cost = (input_tokens * input_rate) + (output_tokens * output_rate)
9     return cost
10
11 # Example usage
12 print(calculate_cost(500, 300, "gpt-4")) # Output: $0.033
```

Note: Adjust rates based on current provider pricing.

Measuring Latency via Code

Latency in LLM API responses is influenced by factors like model size, network conditions, and server load, but can be reduced by using smaller models, optimizing prompts, and choosing nearby server regions.

Code Sample:

```
1 import time
2 import openai
3
4 start_time = time.time()
5 response = openai.Completion.create(model="text-davinci-003", prompt="Hello, world!")
6 end_time = time.time()
7 latency = end_time - start_time
8 print(f"Latency: {latency:.2f} seconds")
```

Evaluating Performance

```
1 import time
2 import concurrent.futures
3
4 def make_api_call(prompt):
5     return openai.Completion.create(model="text-davinci-003", prompt=prompt)
6
7 prompts = ["Hello"] * 10
8 start_time = time.time()
9 with concurrent.futures.ThreadPoolExecutor() as executor:
10     results = list(executor.map(make_api_call, prompts))
11 throughput = len(prompts) / (time.time() - start_time)
12 print(f"Throughput: {throughput:.2f} requests/second")
```

Response Quality: Use metrics like BLEU (translation) or ROUGE (summarization).

Summary

In this lesson, you have learned to:

- e! Define leading LLM providers and distinguish their primary services.
- e! Summarize the main capabilities enabled by modern LLMs.
- e! Interpret the impact of key configuration parameters on model behavior.
- e! Distinguish between major inference techniques used in LLM-based generation.
- e! Utilize practical techniques to reduce token usage and manage context effectively.
- e! Build function-calling workflows using structured outputs in LLMs.